Отчет по практическому заданию №1

Гусейнов А.Э. 681

2 ноября 2026 г.

Содержание

1 Введение **2**

2 Постановка задачи **2**

3 Метод фиктивных областей **2**

4 Разностная схема **3**

5 Метод сопряженных градиентов с диагональным предобуславливанием **3**

6 Параллельная реализация на OpenMP **4**

7 Результаты расчетов **5**

8 Заключение **6**

A Основные фрагменты программы **7**

1 Введение

В работе рассматривается двумерная задача Дирихле для уравнения Пуассона в криволинейной области. Для приближенного решения используется метод фиктивных областей с продолжением по младшему коэффициенту. Полученная дифференциальная задача аппроксимируется на равномерной прямоугольной сетке разностной схемой. Система линейных алгебраических уравнений решается методом сопряженных градиентов с диагональным предобуславливанием. Последовательная программа распараллелена с помощью технологии OpenMP.

Вариант задания: **вариант 7** — квадрат с отсеченной вершиной:

$$D = \{(x, y) : |x| + |y| < 2,\ y < 1\}.$$

Вершины области: $(-2, 0)$, $(0, -2)$, $(2, 0)$, $(1, 1)$, $(-1, 1)$.

2 Постановка задачи

В области $D \subset \mathbb{R}^2$ с кусочно-гладкой границей γ требуется найти функцию $u(x, y)$, удовлетворяющую уравнению Пуассона

$$-\Delta u = f(x, y), \qquad (x, y) \in D, \tag{1}$$

и граничному условию Дирихле

$$u(x, y) = \varphi(x, y), \qquad (x, y) \in \gamma. \tag{2}$$

Для варианта 7:

$$f(x, y) = 1, \qquad \varphi(x, y) = \sin(x + y).$$

3 Метод фиктивных областей

Область D погружается в прямоугольник

$$\Pi = \{(x, y) : A_1 < x < B_1,\ A_2 < y < B_2\}.$$

Для варианта 7: $A_1 = -2$, $B_1 = 2$, $A_2 = -2$, $B_2 = 1$. Фиктивная область $\hat{D} = \Pi \setminus D$. Вводится малый параметр $\varepsilon > 0$. Решается задача

$$-\Delta v + Q(x, y)\big(v - v_0(x, y)\big) = F(x, y), \quad (x, y) \in \Pi \setminus \gamma, \tag{3}$$

$$v(x, y) = v_0(x, y), \quad (x, y) \in \Gamma,$$

где

$$Q(x, y) = \begin{cases} 0, & (x, y) \in D, \\ \varepsilon^{-2}, & (x, y) \in \hat{D}, \end{cases} \qquad F(x, y) = \begin{cases} f(x, y), & (x, y) \in D, \\ 0, & (x, y) \in \hat{D}. \end{cases}$$

В качестве продолжения v_0 используется $\varphi(x, y) = \sin(x + y)$. Параметр ε выбирается как $\varepsilon = h^2$, где $h = \max(h_1, h_2)$ — максимальный шаг сетки.

4 Разностная схема

В прямоугольнике Π вводится равномерная сетка

$$\bar{\omega}_h = \bar{\omega}_1 \times \bar{\omega}_2, \qquad \bar{\omega}_1 = \{x_i = A_1 + ih_1,\ i = 0, \ldots, M\}, \quad \bar{\omega}_2 = \{y_j = A_2 + jh_2,\ j = 0, \ldots, N\},$$

где $h_1 = (B_1 - A_1)/M$, $h_2 = (B_2 - A_2)/N$. Внутренние узлы обозначаются ω_h. Дифференциальное уравнение аппроксимируется разностным уравнением

$$-w_{x\bar{x},ij} - w_{y\bar{y},ij} + Q_{ij} w_{ij} = F_{ij} + Q_{ij} v_0(x_i, y_j), \qquad i = 1, \ldots, M-1, \quad j = 1, \ldots, N-1. \tag{4}$$

Граничные условия аппроксимируются точно:

$$w_{ij} = v_0(x_i, y_j), \qquad (x_i, y_j) \in \Gamma.$$

Коэффициенты Q_{ij} и F_{ij} вычисляются как средние по ячейке Π_{ij}:

$$Q_{ij} = \frac{1}{h_1 h_2} \iint_{\Pi_{ij}} Q(x, y)\, dx\, dy, \qquad F_{ij} = \frac{1}{h_1 h_2} \iint_{\Pi_{ij}} F(x, y)\, dx\, dy.$$

Для варианта 7 доля площади ячейки, попавшая в D, вычислялась приближенно по подсетке 12×12, что позволяет единообразно обрабатывать криволинейные границы. В результате получается система линейных алгебраических уравнений

$$Aw = B \tag{5}$$

с симметричной положительно определенной матрицей A.

5 Метод сопряженных градиентов с диагональным предобуславливанием

Система (5) решается методом сопряженных градиентов. Используется диагональный предобуславливатель

$$(Dw)_{ij} = \left(\frac{2}{h_1^2} + \frac{2}{h_2^2} + Q_{ij} \right) w_{ij}.$$

В задании скалярное произведение и соответствующая ему евклидова норма определены как

$$(u, v)_E = h_1 h_2 \sum_{i=1}^{M-1} \sum_{j=1}^{N-1} u_{ij} v_{ij}, \qquad \|u\|_E = \sqrt{(u, u)_E}.$$

Итерационный процесс:

$$\begin{aligned}
&r^{(0)} = B - Aw^{(0)}, \qquad Dz^{(0)} = r^{(0)}, \qquad p^{(1)} = z^{(0)}, \\
&\alpha_k = \frac{(z^{(k-1)}, r^{(k-1)})}{(Ap^{(k)}, p^{(k)})}, \\
&w^{(k)} = w^{(k-1)} + \alpha_k p^{(k)}, \\
&r^{(k)} = r^{(k-1)} - \alpha_k A p^{(k)}, \\
&Dz^{(k)} = r^{(k)}, \\
&\beta_{k+1} = \frac{(z^{(k)}, r^{(k)})}{(z^{(k-1)}, r^{(k-1)})}, \\
&p^{(k+1)} = z^{(k)} + \beta_{k+1} p^{(k)}.
\end{aligned}$$

Начальное приближение $w^{(0)} = 0$. Критерий остановки:

$$\|w^{(k+1)} - w^{(k)}\|_E < \delta,$$

где $\delta = 10^{-8}$. В отношениях для α_k и β_k общий множитель $h_1 h_2$ сокращается, поэтому в программе для них используются обычные суммы. Критерий остановки, напротив, проверяется в требуемой сеточной норме:

$$\|w^{(k+1)} - w^{(k)}\|_E = \sqrt{h_1 h_2}\,|\alpha_k| \sqrt{\sum_{i,j} (p_{ij}^{(k)})^2} < \delta.$$

Качество итераций дополнительно контролируется функционалом

$$J(w) = \frac{1}{2}(Aw, w)_E - (B, w)_E.$$

Это числовая «энергия» сеточного решения, а не дополнительная неизвестная. Так как A положительно определен, минимум J достигается при $Aw = B$. В программе функционал вычисляется непосредственно по формуле; если его рост превышает оценку погрешности суммирования, решение возвращается к последней итерации с монотонным убыванием, остаток пересчитывается, а метод сопряженных градиентов перезапускается с новым направлением.

6 Параллельная реализация на OpenMP

Последовательная и параллельная версии собраны из одного файла `main.cpp`: без флага `-fopenmp` и с этим флагом соответственно. Распараллелены вычисление Q_{ij}, F_{ij}, правой части B, умножение матрицы на вектор, скалярные произведения и обновление векторов. Для вложенных циклов используется `collapse(2)`, распределение итераций — `schedule(static)`, для сумм — `reduction`. В OpenMP-версии время измеряется функцией `omp_get_wtime()` только внутри итерационного процесса, включая дополнительное умножение на A для проверки J; построение сетки, коэффициентов и правой части в измеряемое время не входит.

Ключевой фрагмент умножения матрицы на вектор:

```
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 1; i <= M - 1; ++i) {
    for (int j = 1; j <= N - 1; ++j) {
        int k = idx(i, j);
        double val = (2.0*inv_h1_2 + 2.0*inv_h2_2 + Q[k]) * u[k];
        if (i > 1) val -= u[idx(i-1,j)] * inv_h1_2;
        if (i < M - 1) val -= u[idx(i+1,j)] * inv_h1_2;
        if (j > 1) val -= u[idx(i,j-1)] * inv_h2_2;
        if (j < N - 1) val -= u[idx(i,j+1)] * inv_h2_2;
        out[k] = val;
    }
}
```

Компиляция:

```
g++ -O0 -std=c++17 main.cpp -o solve
g++ -O0 -fopenmp -std=c++17 main.cpp -o solve_omp
```

Эти команды соответствуют локальному скрипту сборки `build.bat`; параметры сборки исполняемых файлов, использованных для расчетов в кластере, в сохраненных логах не указаны. Для сетки 80×80 запуск выполнялся с 1, 4 и 16 нитями:

```
export OMP_NUM_THREADS=1
./solve_omp 80 80
export OMP_NUM_THREADS=4
./solve_omp 80 80
export OMP_NUM_THREADS=16
./solve_omp 80 80
```

7 Результаты расчетов

В таблице 1 приведены последовательные расчеты, в таблицах 2–4 — ранее сохраненные OpenMP-запуски из папки `result`. Эти файлы получены до исправления критерия остановки и добавления контроля J, поэтому времена, итерации и невязки в таблицах не являются результатами исправленной версии и должны быть пересчитаны для итогового сравнения. Время — время итерационного процесса; невязка исправленной программы выводится в сеточной норме $\|r\|_E$. Число неизвестных равно $(M-1)(N-1)$.

Таблица 1: Архивные последовательные расчеты до исправления критерия остановки

$M \times N$	Итерации	Время, с	$\|r\|_2$
20×20	9	0.000936245	$4.59617 \cdot 10^{-7}$
40×40	7	0.00305351	$1.28974 \cdot 10^{-7}$
80×80	5	0.0104305	$4.31385 \cdot 10^{-6}$

Таблица 2: Архивная масштабируемость OpenMP на сетке 80×80, до исправления

Число нитей	Итерации	Время, с	Ускорение относительно 1 нити
1	5	0.00896651	1.000
4	5	0.00235089	3.814
16	5	0.000963256	9.309

Таблица 3: Архивная масштабируемость OpenMP на сетке 400×600, до исправления

Число нитей	Итерации	Время, с	Ускорение относительно 2 нитей
2	4	0.135180	1.000
4	4	0.0677342	1.996
8	4	0.0345109	3.917
16	4	0.0228812	5.908

В архивных запусках прежней версии программы на сетке 80×80 ускорение относительно одного потока составило 3.814 для 4 потоков и 9.309 для 16 потоков. На

Таблица 4: Архивная масштабируемость OpenMP на сетке 800×1200, до исправления

Число нитей	Итерации	Время, с	Ускорение относительно 4 нитей
4	4	0.288312	1.000
8	4	0.135952	2.121
16	4	0.0587516	4.907
32	4	0.0950004	3.035

сетке 400×600 рост числа потоков с 2 до 16 дал ускорение 5.908, а на сетке 800×1200 наилучшее время получено при 16 потоках. При 32 потоках время увеличилось относительно 16 потоков. Эти измерения характеризуют только прежнюю версию; после исправления критерия остановки и добавления контроля J производительность следует измерить заново. Время разных размеров сетки напрямую сравнивать нельзя.

В этих запусках максимальное число итераций ограничено числом неизвестных. Результаты OpenMP-запусков могут незначительно отличаться по невязке из-за порядка суммирования при параллельной редукции. Значения невязки характеризуют решение дискретной системы с указанным критерием остановки и не являются оценкой ошибки относительно точного решения исходной дифференциальной задачи.

8 Заключение

Реализованы метод фиктивных областей, конечно-разностная схема и решение системы методом сопряженных градиентов с диагональным предобуславливанием. Основные циклы распараллелены с помощью OpenMP. Исправленная программа использует норму из задания и контролирует монотонность функционала J, при необходимости перезапуская итерационный процесс. Ранее сохраненные измерения относятся к предыдущей версии программы; их необходимо повторить, чтобы корректно сравнить производительность и невязку исправленной версии.

Список литературы

[1] А.Н. Тихонов, А.А. Самарский. Уравнения математической физики. М.: Наука, 1977.

[2] Г.И. Марчук. Методы вычислительной математики. М.: Наука, 1989.

[3] П.Н. Вабищевич. Метод фиктивных областей в задачах математической физики. 2017.

[4] А.А. Самарский. Теория разностных схем. М.: Наука, 1989.

[5] А.А. Самарский, А.В. Гулин. Численные методы. М.: Наука, 1989.

[6] В.Б. Андреев. Численные методы. М.: Макс Пресс, 2013.

[7] А.С. Антонов. Технологии параллельного программирования MPI и OpenMP. М.: Изд-во МГУ, 2012.

[8] OpenMP Architecture Review Board. OpenMP Application Programming Interface. https://www.openmp.org/.

A Основные фрагменты программы

Полный код программы прилагается отдельно. Ниже приведен фрагмент, задающий геометрию варианта 7 и функцию проверки точки внутри области.

```
const vector<Point> DOMAIN = {
    {-2.0, 0.0},
    { 0.0, -2.0},
    { 2.0, 0.0},
    { 1.0, 1.0},
    {-1.0, 1.0}
};

double f(double x, double y) { return 1.0; }
double phi(double x, double y) { return sin(x + y); }
double v0(double x, double y) { return sin(x + y); }

bool insideDomain(double x, double y) {
    bool inside = false;
    int n = (int)DOMAIN.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        const Point& pi = DOMAIN[i];
        const Point& pj = DOMAIN[j];
        bool intersect = ((pi.y > y) != (pj.y > y)) &&
            (x < (pj.x - pi.x) * (y - pi.y) / (pj.y - pi.y) + pi.x);
        if (intersect) inside = !inside;
    }
    return inside;
}
```